

Smart Dispatch IA

Informe Final - Orquestacion Agentica Ciclica y Memoria Persistente

Recurso	Link
Aplicacion en vivo	https://smart-dispatch-q4xk.onrender.com
Repositorio GitHub	https://github.com/rossanny25/smart-dispatch
Demo Docker local	http://127.0.0.1:8050
Acceso demo	Usuario admin / clave smart2026AI
Guia de ejecucion	docs/runbook.md
Guia Ollama local opcional	docs/ollama-local-demo.md
Evidencia de sesion	docs/usage-session-log.md

Nota: la aplicacion publicada usa Render Free. Si la instancia estuvo inactiva, la primera carga puede demorar cerca de 50 segundos o mas mientras el servicio despierta.

Checklist de validacion

Requisito del proyecto	Estado	Evidencia
Aplicacion real funcionando	Cumplido	Demo en Render y Docker local
Links en primera pagina	Cumplido	Tabla inicial
Arquitectura y UML	Cumplido	Diagramas incluidos
Tecnologias justificadas	Cumplido	Tabla tecnica
Capturas y log real	Cumplido	Secciones 9 y 10
UX/UI Nielsen	Cumplido	Seccion 11
Ciberseguridad	Cumplido	Seccion 12
Uso de IA y reflexion LLM/SLM	Cumplido	Secciones 13 y 14

1. Resumen ejecutivo

Smart Dispatch IA es un prototipo funcional de asistencia al despacho tecnico en servicios de campo. El proyecto convierte un modelo conceptual de orquestacion agentica y memoria persistente en una aplicacion web publicada, ejecutable con Docker y documentada con evidencia tecnica reproducible.

El sistema ayuda a decidir que tecnico conviene asignar a una orden de trabajo. Para hacerlo, separa el problema en etapas: captura de informacion, analisis de requerimientos, planificacion, evaluacion de restricciones, scoring, confianza y aprendizaje. La aplicacion no reemplaza al despachador: funciona como soporte a la decision y conserva evidencia de por que se recomienda un tecnico.

La evolucion principal frente al trabajo teorico inicial es que la orquestacion deja de ser una descripcion general y pasa a estar formalizada como una maquina de estados deterministica. Las reglas duras se aplican antes del ranking, la funcion objetivo esta versionada, la confianza se calcula separada del puntaje y la memoria persistente queda tratada como evidencia controlada.

2. Contexto y problema

En operaciones de campo, un despachador suele asignar tecnicos con informacion incompleta: tipo de incidente, zona, urgencia, habilidades requeridas, disponibilidad, carga de trabajo, distancia, clima, trafico y experiencia historica. Una mala asignacion puede causar demoras, incumplimiento de SLA, sobrecarga de tecnicos o decisiones poco explicables.

El problema no es solo seleccionar el tecnico mas cercano. Tambien hay restricciones que no deberian negociarse: disponibilidad, certificaciones, turno, jornada maxima, limite de conduccion y elementos de seguridad. Por eso el sistema primero determina factibilidad, luego rankea candidatos y finalmente expone evidencia para que el humano decida.

3. Evolucion desde el trabajo de medio ciclo

Observacion del feedback	Evolucion implementada
Orquestador ambiguo	Maquina de estados deterministica controlada por DispatchOrchestrator.
Memoria persistente poco formalizada	Separacion entre seeds, runtime SQLite y memoria de aprendizaje legacy.
Funcion objetivo descriptiva	Scoring con componentes, pesos, penalizaciones y version de configuracion.
Aprendizaje generico	Aprendizaje incremental/evidencial, sin prometer fine-tuning.
Falta de metricas concretas	KPIs y evidencia requerida documentados para evaluacion futura.
Incertidumbre incompleta	Confianza independiente, warnings y explicaciones estructuradas.
Falta de app real	Render, Docker, screenshots, API evidence y logs reales.

4. Arquitectura general

La arquitectura objetivo es un monolito modular hexagonal con pipeline deterministico. Se eligio monorepo porque el frontend es estatico y FastAPI puede servir API y UI desde el mismo proceso. Esto reduce friccion operativa: un repositorio, un README, un comando Docker y una ruta clara de despliegue.

Despachador

- > Browser UI
- > FastAPI HTTP Adapter (/api/v1 y /api)
- > Application Commands
- > DispatchOrchestrator
- > Analyze Adapter (deterministico u Ollama local opcional)
- > Domain Policies (eligibility, scoring, confidence)
- > Unit Of Work / SQLite Repositories
- > SQLite (runs, snapshots, stages, transitions)

- frontend/: interfaz HTML, CSS y JavaScript vanilla.
- app/api/v1: API canonica versionada.
- app/application: comandos y casos de uso.
- app/domain: politicas puras de negocio.
- app/adapters: persistencia, legacy API y etapas deterministicas.
- data/seeds: datos reproducibles de demo.
- docs: informe, diagramas, evidencia y runbook.

5. Orquestacion agentica ciclica

El sistema modela el ciclo de despacho como una secuencia de estados. La pieza central es DispatchOrchestrator: los agentes no pueden avanzar estados por su cuenta. Esto reemplaza el orquestador ambiguo por un mecanismo auditable.

```
[*] -> CAPTURE -> ANALYZE -> PLAN -> EVALUATE
EVALUATE -> WAIT_FOR_DECISION
EVALUATE -> NO_FEASIBLE_CANDIDATES
CAPTURE/ANALYZE/PLAN/EVALUATE -> FAILED
```

Etapas	Responsabilidad
CAPTURE	Normaliza y valida la informacion de entrada.
ANALYZE	Deriva categoria, prioridad, SLA, certificaciones y duracion estimada.
PLAN	Aplica reglas duras y calcula ranking solo para candidatos elegibles.
EVALUATE	Agrega confianza, advertencias y explicacion sin cambiar el ranking.
WAIT_FOR_DECISION	Espera la decision humana del despachador.

6. UML principal

```
WorkOrder 1 -> many DispatchRun
DispatchRun 1 -> many RunSnapshot
DispatchRun 1 -> many StageExecution
DispatchRun 1 -> many StateTransition
DispatchRun 1 -> many EligibilityCandidate
EligibilityCandidate 0..1 -> 0..1 ScoredTechnician
ScoredTechnician 0..1 -> 0..1 ConfidenceOutput
Technician 1 -> many EligibilityCandidate
```

El modelo conserva la separacion entre orden, corrida de despacho, snapshots, ejecuciones por etapa, transiciones de estado, candidatos elegibles, puntajes y salida de confianza. Esta separacion permite explicar cada recomendacion sin mezclar reglas duras, scoring y memoria.

7. Tecnologias usadas

Tecnologia	Uso	Justificacion
Python 3.12	Backend principal	Lenguaje claro y buen ecosistema web/testing.
FastAPI	API HTTP	Contratos claros, OpenAPI y soporte ASGI.
Pydantic v2	Validacion	Modelos estrictos y rechazo de campos desconocidos.
SQLAlchemy Core	Persistencia	SQL explicito sin acoplar dominio a ORM.
Alembic	Migraciones	Control de evolucion del esquema SQLite.
SQLite	Base local	Suficiente para un MVP single-user y despliegues livianos.
HTML/CSS/JS	Frontend	Interfaz simple, auditable y sin build complejo.
Docker/Compose	Ejecucion	Reproducibilidad local y deploy con Dockerfile.
Render Free	Publicacion	Link publico evaluable.
pytest	Verificacion	Pruebas unitarias, integracion y contrato.
BMad Method	Especificacion	PRD, arquitectura, epicas, historias y trazabilidad.

8. Reglas duras, scoring y memoria

El prototipo distingue entre restricciones duras y criterios de optimización. Un técnico que falla una restricción dura no puede recibir puntaje objetivo. Esta decisión evita que una puntuación alta o una preferencia aprendida oculte una violación operativa.

- Técnico disponible.
- Certificaciones requeridas.
- Turno vigente.
- Jornada máxima.
- Límite de conducción.
- EPP requerido.

$$\text{score} = 0.35 \cdot \text{SLA} + 0.25 \cdot \text{proximidad} + 0.20 \cdot \text{balance_carga} + 0.10 \cdot \text{calidad} + 0.10 \cdot \text{memoria} - \text{penalizaciones}$$

- Técnicos demo: data/seeds/technicians.json.
- Órdenes demo: data/seeds/orders.json.
- Memoria inicial: data/learning_store.json.
- Runtime SQLite local: data/smart_dispatch.db.
- Runtime Docker: volumen smart_dispatch_data.
- Reset operativo: POST /api/reset o docker compose down -v.
- Acceso demo: usuario admin, clave smart2026AI.

El sistema incluye usuarios persistidos en SQLite, roles básicos y login con cookie de sesión firmada. Existen paneles admin para listar, crear y editar usuarios y técnicos. Los técnicos operativos se inicializan desde seeds solo cuando la tabla está vacía; luego se editan en SQLite y afectan la siguiente simulación de despacho. Las fichas operativas incluyen contacto, documentos y notas de auditoría. Las órdenes demo también se inicializan en SQLite desde seeds, y los cierres de despacho crean visitas visibles en el Calendario y en el Mapa Operativo local.

9. Capturas del frontend

The screenshot displays the 'Smart Dispatch IA' dashboard. At the top, the header includes the logo 'Smart Dispatch IA', version 'V2.0', and the tagline 'ORQUESTACIÓN CÍCLICA Y MEMORIA PERSISTENTE'. To the right of the header, there are three dropdown menus for environmental factors: 'Clima' (set to 'Soleado (Acelerador)'), 'Tráfico' (set to 'Normal'), and 'Señal GPS' (set to 'Óptima (Acelerador)'). A 'Reset' button is also present. Below the header, a navigation bar contains several buttons: 'Nueva Solicitud', 'Cola de Órdenes', 'Recorrido Guiado', 'Consola y Recomendación', 'Técnicos', 'Mapa', 'Calendario', 'Memoria', and 'Administración'. The main content area features a form titled 'Nueva Solicitud de Servicio'. This form has a text input field for 'Descripción de la Avería (Lenguaje Natural)' with the example text 'Ej: Urgente fuga de gas en la sucursal Palermo, requiere matriculado para reparar regulador general...'. Below this, there are two input fields: 'Dirección Física' (containing 'Av. Santa Fe 3400') and 'Zona' (a dropdown menu set to 'Palermo'). At the bottom of the form is a button labeled 'Ingresar y Procesar con IA'.

Figura 1. Dashboard inicial con ordenes, tecnicos y controles de contexto.

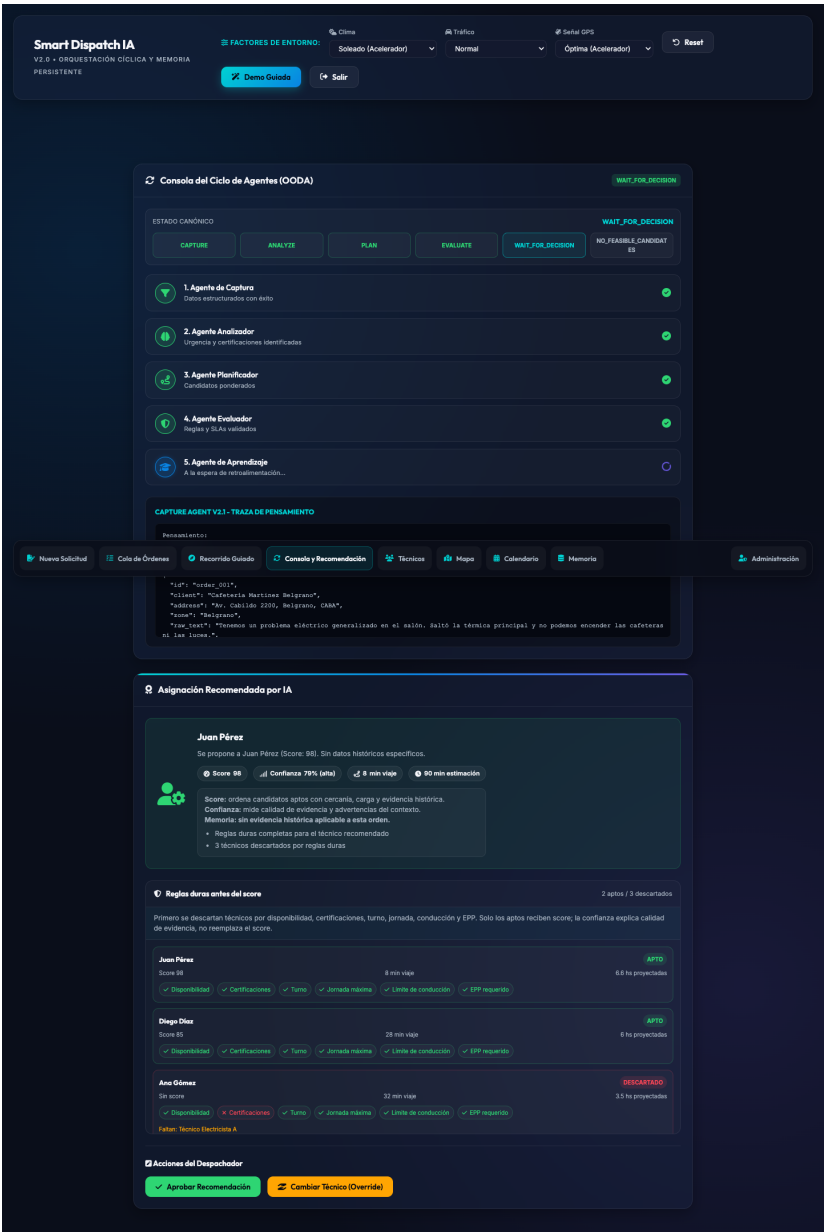


Figura 2. Resultado de simulacion con ciclo agentico, estados canonicos, reglas duras y recomendacion.

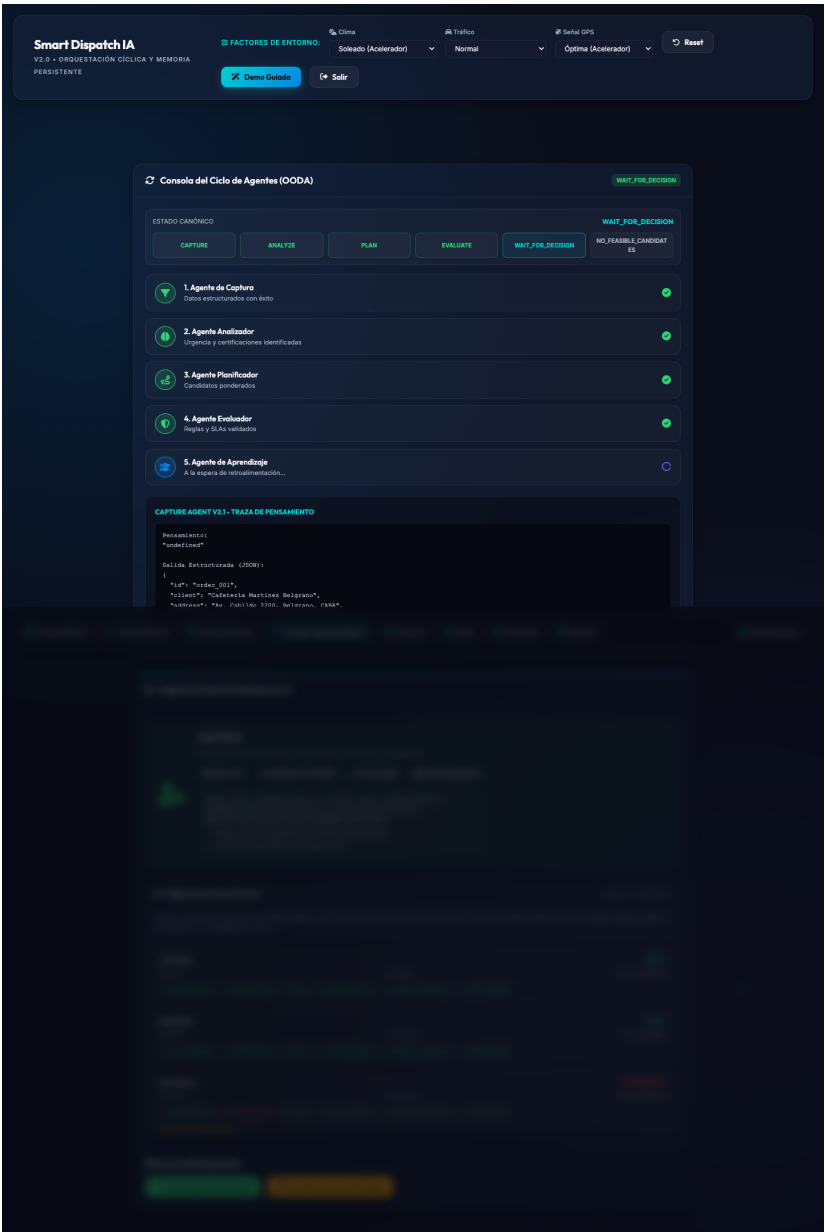


Figura 3. Aprobacion de recomendacion y modal de cierre.

Smart Dispatch IA

V2.0 • ORQUESTACIÓN CÍCLICA Y MEMORIA

PERSISTENTE

FACTORES DE ENTORNO:

Clima

Soleado (Acelerador)

Tráfico

Normal

Señal GPS

Óptima (Acelerador)

Reset

Demo Guiada

Salir

Nueva Solicitud

Cola de Órdenes

Recorrido Guiado

Consola y Recomendación

Técnicos

Mapa

Calendario

Memoria

Administración

Calendario de Visitas

1 VISITA

Técnico

Todos los técnicos

1 completadas. La agenda se alimenta al cerrar despachos reales.

Ana Gómez

0 completadas - 0 min

Sin visitas cerradas

Carlos Rodríguez

0 completadas - 0 min

Sin visitas cerradas

Diego Díaz

0 completadas - 0 min

Sin visitas cerradas

Juan Pérez

1 completada - 1.5 hs

Última: mié, 19 ago

Sofía Torres

0 completadas - 0 min

Sin visitas cerradas

Técnico

Cliente

Dirección

Zona

Inicio

Seleccionar técnico

Cliente / Sucursal

Dirección operativa

Palermo

dd/mm/aaaa, --:--

Duración min

90

Programar visita

mié, 19 ago

Juan Pérez - 1 visita

mié, 19 ago

04:11 p. m. - 05:41 p. m.

Cafetería Martínez Belgrano • Electricidad

Juan Pérez

Belgrano

90 min

Av. Cabildo 2200, Belgrano, CABA

programada

en_curso

cancelada

COMPLETADA

Figura 4. Orden completada y aprendizaje registrado.

Smart Dispatch IA - Informe Final

Pagina 9

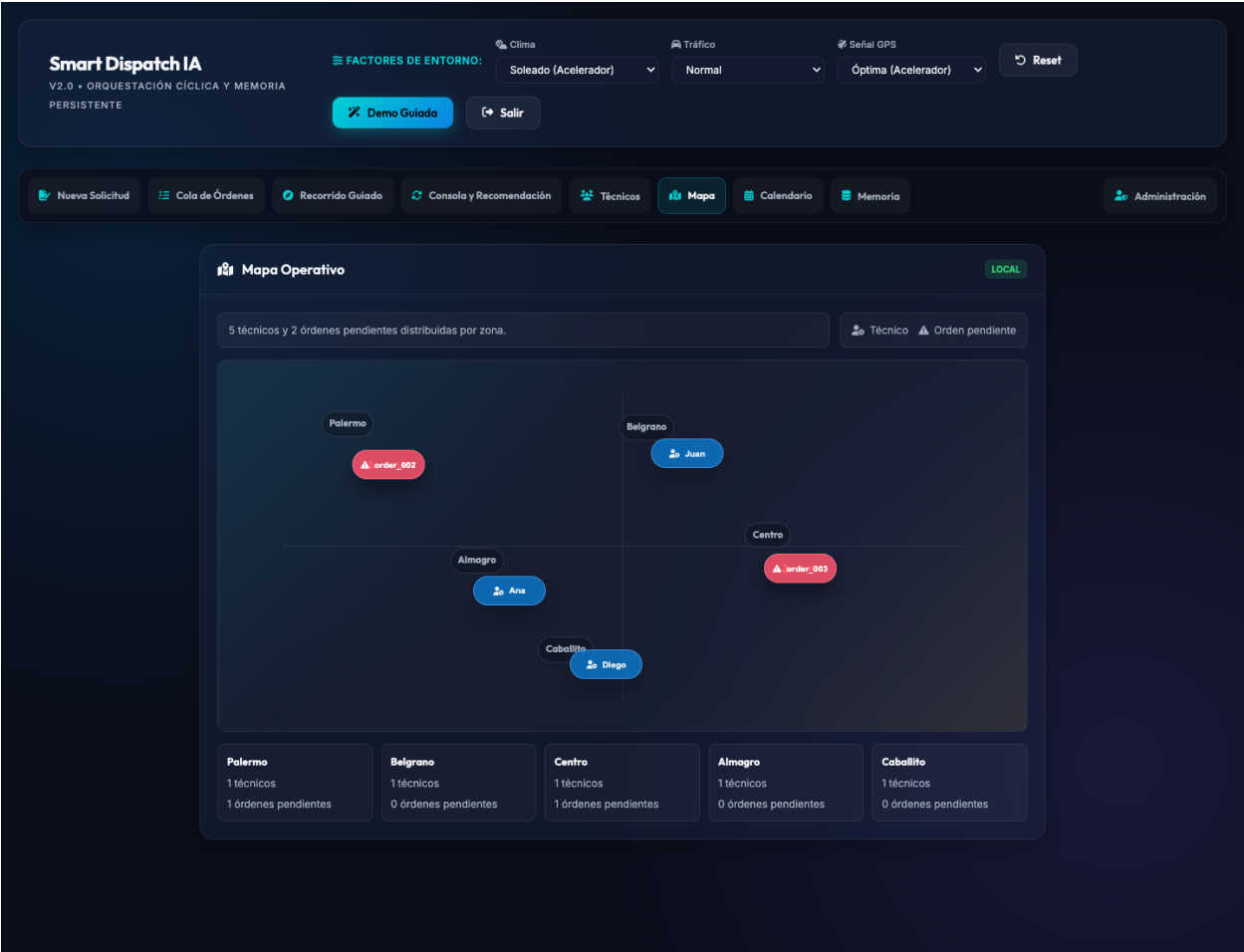


Figura 5. Mapa operativo local con ordenes y tecnicos por zona.

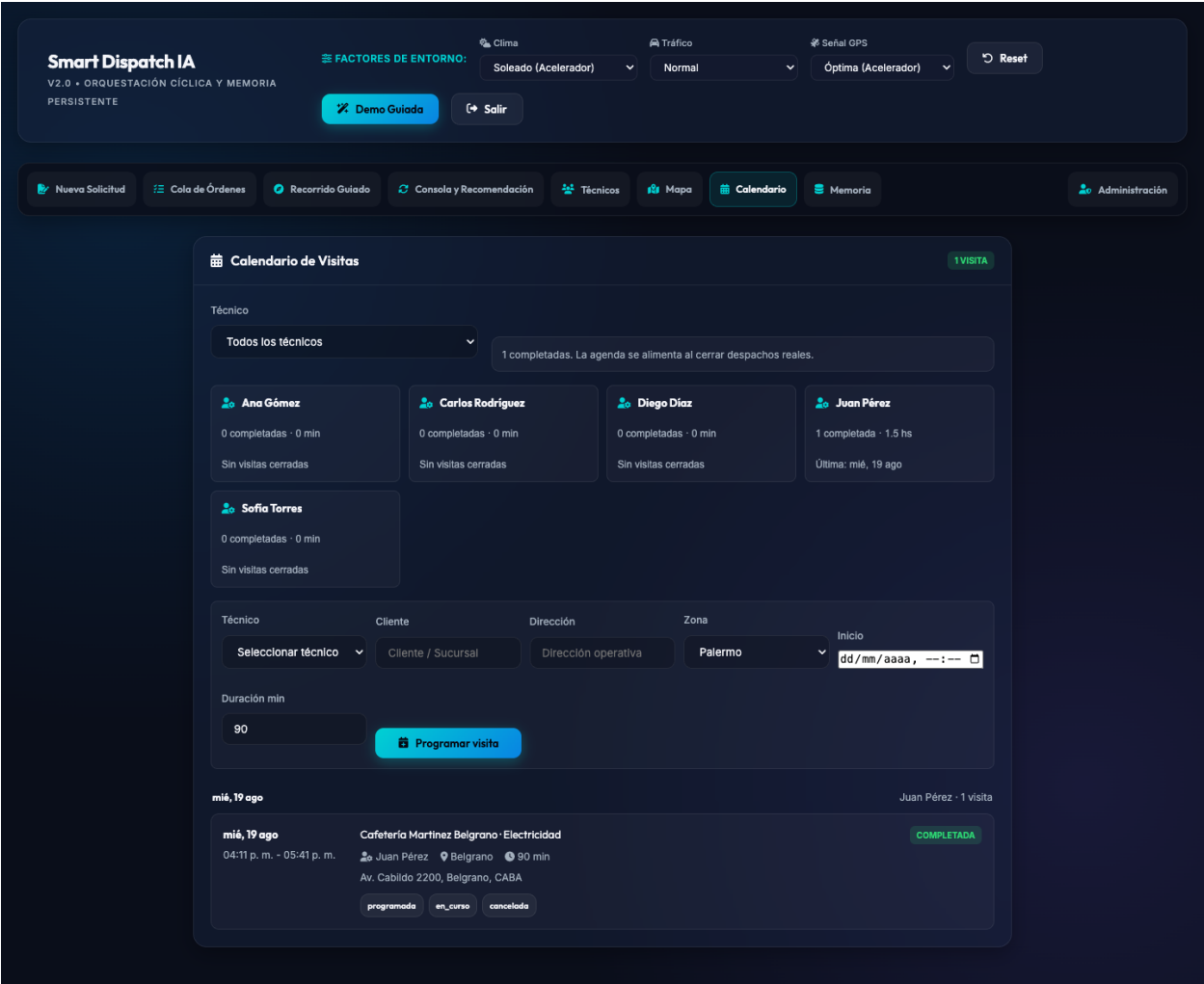


Figura 6. Calendario de visitas agrupado por tecnico y estado.

Administración

Gestión de usuarios, roles y técnicos de campo

X

Usuarios y roles

Técnicos

Administración de Técnicos

CREAR

Nombre

Estado operativo

Disponible

Zona base

Palermo

Certificaciones

Gasista Matriculado, Técnico HVAC

Inicio de turno

08:00 a.m.

Fin de turno

05:00 p.m.

Carga actual (hs)

0

Calificación

4.5

EPP disponible

Arnés, Casco dieléctrico

GPS latitud

-34.6037

GPS longitud

-58.3816

Teléfono

+54 11 5555-0101

Email

tecnico@empresa.test

Documentos

Matricula, Seguro ART, Licencia

Auditoría

Nota Interna opcional

Guardar técnico

Limpiar

Figura 7. Administracion de tecnicos con contacto, documentos y auditoria.

Smart Dispatch IA

V2.0 • ORQUESTACIÓN CÍCLICA Y MEMORIA

PERSISTENTE

FACTORES DE ENTORNO:

Clima

Soleado (Acelerador)

Tráfico

Normal

Señal GPS

Óptima (Acelerador)

Reset

Demuestra Guía

Salir

Consola del Ciclo de Agentes (OODA)

NO_FEASIBLE_CANDIDATES

ESTADO CANÓNICO

CAPTURE

ANALYZE

PLAN

EVALUATE

WAIT_FOR_DECISION

NO_FEASIBLE_CANDIDATES

1. Agente de Captura

Datos estructurados con éxito

2. Agente Analizador

Urgencia y certificaciones identificadas

3. Agente Planificador

Candidatos ponderados

4. Agente Evaluador

Nueva Solicitud

Cola de Órdenes

Recorrido Guiado

Consola y Recomendación

Técnicos

Mapa

Calendario

Memoria

Administración

Sin asignación factible para aprender

CAPTURE AGENT V2.1- TRAZA DE PENSAMIENTO

Pensamiento:

"undefined"

Salida Estructurada (JSON):

```
{
  "id": "order_003",
  "title": "Data Center Puerto Madero",
  "address": "Av. Alicia Moreau de Justo 1848, Puerto Madero, CABA",
  "zone": "Centro",
  "raw_text": "Incidente crítico en sala eléctrica de media tensión. Se requiere operador certificado MT2 para intervenir tablero principal energizado."
}
```

Asignación Recomendada por IA

0 aptos / 5 descartados

Reglas duras antes del score

Primero se descartan técnicos por disponibilidad, certificaciones, turno, jornada, conducción y EPP. Solo los aptos reciben score; la confianza explica calidad de evidencia, no reemplaza el score.

Ana Gómez

Sin score

16 min viaje

DESCARTADO

3.3 hs proyectadas

Disponibilidad

Certificaciones

Turno

Jornada máxima

Límite de conducción

EPP requerido

Faltan: Operador Certificado MT2

Carlos Rodríguez

Sin score

28 min viaje

DESCARTADO

5.5 hs proyectadas

Disponibilidad

Certificaciones

Turno

Jornada máxima

Límite de conducción

EPP requerido

Faltan: Operador Certificado MT2

Diego Díaz

Sin score

28 min viaje

DESCARTADO

6 hs proyectadas

Disponibilidad

Certificaciones

Turno

Jornada máxima

Límite de conducción

EPP requerido

Faltan: Operador Certificado MT2

Sin candidatos factibles

No se fuerza una recomendación porque ningún técnico cumple todas las restricciones duras.

Figura 8. Escenario NO_FEASIBLE_CANDIDATES sin recomendacion forzada.

10. Log de sesion real

Campo	Valor
Fecha	2026-08-19
Runtime	Docker Compose
URL	http://127.0.0.1:8050
Comando	docker compose up --build
Objetivo	Demostrar la version actual con estados canonicos, calendario, mapa, admin operativo y escenario no factible.

- Se inicio la aplicacion Dockerizada.
- Se abrio el frontend y se verifiko /api/technicians.
- Se ejecuto una simulacion de despacho.
- Se aprobo la recomendacion.
- Se completo el servicio y se registro aprendizaje.
- Se revisaron Mapa Operativo, Calendario y Administracion de tecnicos.
- Se ejecuto el escenario NO_FEASIBLE_CANDIDATES.
- Se exportaron logs Docker/Uvicorn.

Resultado observado: orden Cafeteria Martinez Belgrano, categoria Electricidad, prioridad 4, tecnico recomendado Juan Perez, score visible 98, viaje 8 minutos, duracion estimada 90 minutos y estado final completada. Tambien se observo NO_FEASIBLE_CANDIDATES para la orden Data Center Puerto Madero, sin forzar recomendacion.

```
Uvicorn running on http://0.0.0.0:8050
GET / HTTP/1.1 200 OK
GET /api/technicians HTTP/1.1 200 OK
GET /api/orders HTTP/1.1 200 OK
POST /api/dispatch/simulate HTTP/1.1 200 OK
POST /api/dispatch/confirm HTTP/1.1 200 OK
```

11. Autoevaluacion UX/UI con Nielsen

Heuristica	Evaluacion	Mejora
Visibilidad del estado	Buena: etapas, recomendacion y estados canonicos visibles.	Conseguir visibilidad/transiciones reales de DispatchRun.
Relacion con el mundo real	Buena: orden, tecnico, zona, prioridad, mapa y wait.	Mantener labels compactos para SLA, reglas y confianza.
Control del usuario	Buena para MVP: aprobar/cambiar, resetear y agregar ordenes.	Agregacion de ordenes canonica completa.
Consistencia	Buena: paneles y estados uniformes.	Unificar errores API en frontend.
Prevencion de errores	Media: backend valida mas que UI.	Validar antes de enviar.
Reconocimiento	Buena: ordenes y tecnicos visibles.	Mantener contexto seleccionado.
Recuperacion de errores	Media: API tipada y no factible visible.	Mejorar validacion de formularios y retry.
Ayuda/documentacion	Buena: README, runbook e informe.	Agregar panel breve dentro de la app.

12. Log de ciberseguridad

Riesgo	Mitigacion actual	Pendiente
Exposicion accidental	Default local 127.0.0.1; Docker explicito en 8050; logins Single-politicas de red.	Hit Single-politicas de red.
Cuentas operativas	Roles admin/tecnico/dispatcher, rutas admin protegidas; Asistiendo a politicas de uso productivo.	Asistiendo a politicas de uso productivo.
Datos sensibles	Evidencia demo y recomendacion por zona.	Politica para datos reales.
JSON malformado/grande	/api/v1 limita 1 MiB y usa errores tipados.	Migrar rutas legacy restantes.
Excepciones inseguras	Errores conocidos se mapean a respuestas estables.	Politica productiva completa.
Drift de dependencias	pyproject.toml, uv.lock y Docker pinnean dependencias.	Escaneo de vulnerabilidades.
Migraciones fallidas	Startup fail-closed y backups SQLite.	Retencion/exportacion productiva.
Assets externos	Aceptable en prototipo.	Vendorizacion futura.

13. Uso de IA en co-work

- Interpretar feedback tecnico y convertirlo en tareas implementables.
- Crear PRD, arquitectura, epicas e historias con BMad.
- Implementar contratos, politicas, persistencia y pruebas.
- Dockerizar la aplicacion.
- Preparar documentacion tecnica, evidencia y artefactos de revision.
- Comparar opciones de deploy y publicacion.

La IA tambien tuvo limites: necesito verificacion real para no asumir que dependencias, Docker o SSH funcionaban; la publicacion final dependio de acciones humanas; y las capturas/logs tenian que salir de una ejecucion real, no de una descripcion.

14. Reflexion sobre integracion de LLM o SLM local

La integracion local de un LLM o SLM se implementa como adaptador opcional de ANALYZE, apagado por defecto. Su funcion es leer texto libre del incidente y proponer campos estructurados: categoria, prioridad, certificaciones, SLA y duracion estimada.

Ollama puede activarse solo en entorno local con SMART_DISPATCH_ANALYZE_ADAPTER=ollama. El despliegue Render no configura Ollama y conserva el adaptador deterministico. Si Ollama no esta disponible, demora demasiado o devuelve JSON invalido, la aplicacion vuelve al analizador deterministico.

La guia operativa del demo local esta documentada en docs/ollama-local-demo.md e incluye los comandos para host local, Docker Compose con servicio Ollama y Docker apuntando al Ollama del host.

- No deberia avanzar estados, saltar reglas duras, seleccionar tecnico final, escribir memoria directamente ni inventar evidencia.
- Su salida deberia pasar por contratos Pydantic. Si no valida, el sistema debe rechazarla como salida invalida de etapa.
- Ventajas: privacidad, demo offline, menor dependencia cloud y buen ajuste para Ollama.
- Limitaciones: menor calidad potencial, dependencia de hardware, latencia, pruebas contra alucinaciones y mantenimiento.

15. Despliegue, roadmap y conclusiones

La aplicacion esta publicada en <https://smart-dispatch-q4xk.onrender.com> y el repositorio esta en <https://github.com/rossanny25/smart-dispatch>. Localmente se ejecuta con docker compose up --build y se abre en <http://127.0.0.1:8050>.

- No hay registro publico de usuarios ni recuperacion de clave por email.
- Las rutas visibles de la UI todavia combinan /api/v1 con rutas de compatibilidad mientras se migra la decision humana canonica.
- La UI legacy todavia muestra algunas trazas descriptivas.
- No se implementa aprendizaje semantico completo de produccion.
- No se garantiza persistencia productiva en hosting gratuito.
- El mapa es operativo/esquematico local; no usa proveedor GIS externo.
- Clima, trafico y GPS son factores simulados, no integraciones reales.

Roadmap recomendado: decision humana y outcome completo sobre /api/v1; conexion de estados visibles con DispatchRun real y revision persistida; escrituras admin dentro del Unit of Work principal; memoria episodica con comparativas memoria on/off; accesibilidad WCAG; autenticacion ampliada solo si evoluciona a uso multiusuario.

Smart Dispatch IA consolida una idea conceptual en una aplicacion real, publicada, ejecutable y documentada. El sistema presenta orquestacion deterministica, reglas duras, scoring, confianza, persistencia, pruebas, Docker, repositorio publico y evidencia de uso.

El aporte principal es mostrar como un sistema agentico puede mantenerse controlado. Los agentes producen evidencia, pero no gobiernan el estado. La memoria puede informar decisiones, pero no reemplaza restricciones de seguridad. La IA puede colaborar en el analisis, pero la aplicacion conserva mecanismos deterministas para que el resultado sea auditable.